

Gemma4 Elderly Care Assistant 技术报告

一、项目概述

本项目面向独居老人安全照护场景，构建一个多模态 AI 监测与主动干预系统。系统通过浏览器摄像头或上传视频检测跌倒，通过麦克风语音确认老人是否安全，并把视觉、语音、无响应时长等上下文统一交给 Gemma 4 决策层，输出风险等级、风险分数、行动建议、报警状态和可解释原因。项目解决的真实痛点是：老人跌倒后可能无法主动呼救，单一视觉检测容易误报，而家属或护理人员需要一个更可信、更可解释的综合判断结果。

二、赛道与提交定位

推荐赛道为 D SocialGood。项目直接服务老人居家安全、跌倒救援和家属通知，是典型社会公益与民生安全方向。同时项目也包含多模态能力，可在 README 中说明具备 Multimodal 特征，但官方目录建议放入 submissions/2026/D/Gemma4-Elderly-Care-Assistant。

三、模型选型理由

本项目采用 **Gemma 4 2B** 作为默认决策模型，并通过 **Ollama 本地部署** 运行。

选择 Gemma 4 2B 的主要原因如下：

- 边缘部署能力 (Edge AI)**
 - 老人居家安全监测场景对隐私要求较高。
 - Gemma 4 2B 可以在普通笔记本电脑或小型边缘设备上运行，减少敏感数据上传云端的需求。
- 低延迟实时决策**
 - 跌倒检测和紧急救援属于实时场景。
 - Gemma 4 2B 能够快速完成风险分析与动作决策，满足秒级响应需求。
- 多模态决策能力**
 - 系统需要综合视觉检测结果、语音确认结果以及上下文状态进行统一判断。
 - Gemma 4 作为决策层能够生成结构化风险评估与行动建议。
- 可扩展性**
 - 当前代码默认使用 gemma4:2b。
 - 模型名称通过环境变量配置，可根据硬件条件切换到更高规格的 Gemma 4 模型。

因此，Gemma 4 2B 在性能、资源占用、隐私保护和演示稳定性之间取得了较好的平衡，适合作为本项目的默认部署方案。

四、Gemma 4 核心调用逻辑

Gemma 4 是本项目的核心决策引擎。

系统首先通过视觉模块和语音模块获取检测结果：

- Vision Service 输出跌倒检测状态和置信度；
- Speech Service 输出语音转写内容和用户意图；
- Context 信息记录无响应时长、历史风险状态等上下文信息。

随后由 DecisionService 统一构建决策上下文：

</> JSON

```
{
  "vision": {...},
  "speech": {...},
  "context": {...}
}
```

DecisionService 调用 Ollama Chat API：

<http://127.0.0.1:11434/api/chat>

默认模型：
gemma4:2b

Gemma 4 根据多模态输入生成统一决策结果：
</> JSON

```
{
  "risk_level": "high",
  "risk_score": 92,
  "action": "trigger_emergency_alert",
  "emergency_alert": true,
  "reason": "Fall detected and no user response"
}
```

Diagram 1: Gemma 4 Decision Flow

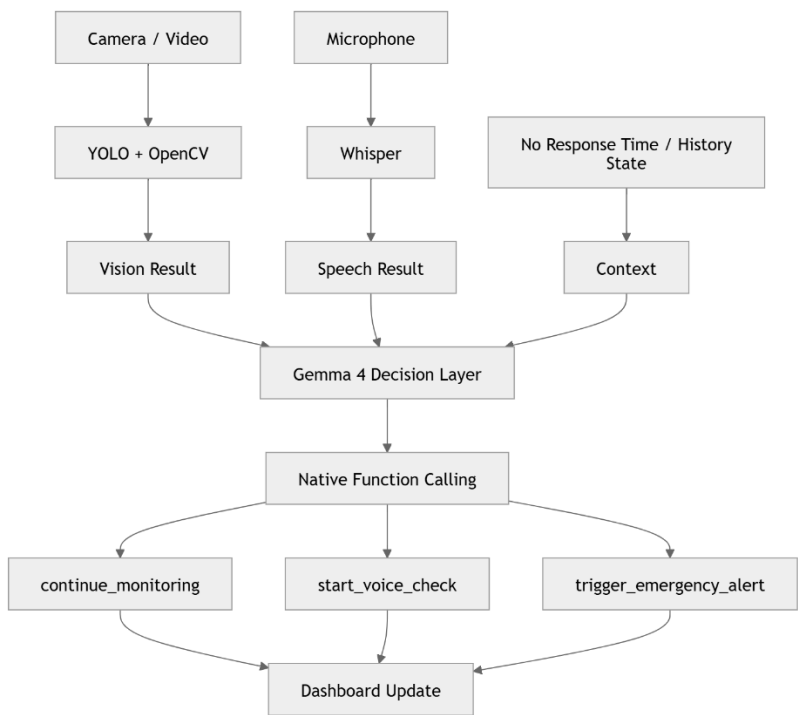


图 1 展示了系统如何将视觉、语音和上下文信息输入 Gemma 4，并通过 Native Function Calling 映射到具体安全动作。

五、Native Function Calling 设计

为满足 Gemma 4 大赛对 Native Function Calling 的要求，本项目将系统动作抽象为三个原生工具：

1. continue_monitoring

适用于：

- 正常活动
- 无风险状态
- 用户确认安全

动作：

继续监测

2. start_voice_check

适用于：

- 疑似跌倒
- 中等风险
- 检测结果存在不确定性

动作:

主动发起语音确认

例如:

您还好吗?

是否需要帮助?

3. trigger_emergency_alert

适用于:

- 高置信度跌倒
- 用户主动求助
- 长时间无响应

动作:

触发紧急报警

系统将上述工具定义为 Gemma 4 可调用的 Native Function。

Gemma 4 不仅生成自然语言解释，还能够输出结构化动作决策，并映射到系统预定义的 Native Function，实现：

观察 → 分析 → 决策 → 动作映射

从而完成完整的智能干预流程。

六、多模态处理架构

本项目采用典型的 Multimodal AI 架构。

视觉模态 (Vision)

视觉模块负责:

- 摄像头实时检测
- 视频检测
- 跌倒识别

主要技术:

- YOLO
- OpenCV

项目同时包含跌倒检测模型训练与验证代码，用于针对老人跌倒场景优化视觉识别效果。相关代码包括:

- yolo/train_urfall_finetune.py
- yolo/evaluate_video_validation.py

训练得到的模型权重用于系统中的跌倒检测推理流程。

输出:

</> JSON

```
{
  "fall_detected": true,
  "confidence": 0.93
}
```

语音模态 (Speech)

语音模块负责：

- 麦克风录音
- Whisper 转写
- 关键词意图识别

识别类别：

safe

help

none

输出：

</> JSON

```
{  
  "transcript": "Help me",  
  "intent": "help"  
}
```

决策模态 (Decision)

Gemma 4 接收视觉模块、语音模块以及系统上下文信息，统一完成风险分析与动作决策。

主要输出：

- 风险等级 (Risk Level)
- 风险分数 (Risk Score)
- 行动建议 (Action)
- 报警状态 (Emergency Alert)
- 决策原因 (Reason)

七、系统架构

Diagram 2: Overall System Architecture

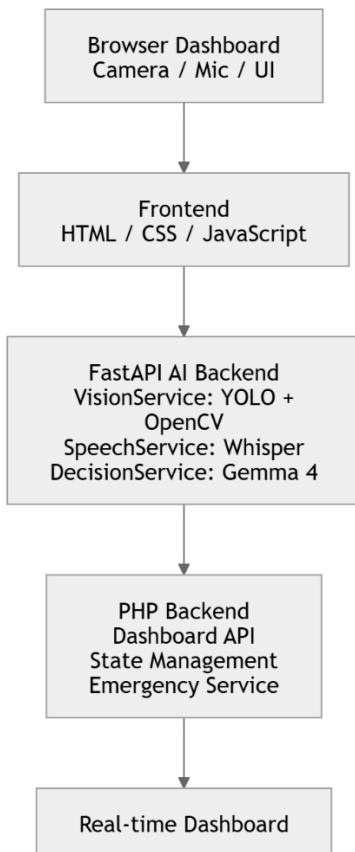


图 2 展示了系统从浏览器端采集数据，到 FastAPI AI 后端推理，再到 PHP 后端同步状态并更新 Dashboard 的整体架构。

八、边缘部署 (Edge Deployment)

本项目支持边缘部署 (Edge AI)。

部署方式：

Browser

FastAPI

Ollama

Gemma 4

该部署方式无需依赖外部云端推理服务，适用于家庭照护、养老机构和网络条件受限环境。

九、风险控制与兜底策略

考虑现场演示和真实照护系统稳定性，DecisionService 保留规则兜底。如果 Ollama 未启动、Gemma 4 模型未下载、请求超时或返回格式异常，系统会自动使用本地规则输出风险结果，并在结果中记录 `gemma_error`。这样既满足 Gemma 4 调用逻辑，又避免模型服务不可用导致 Dashboard 完全失效。

十、核心代码清单

必须提交并重点展示的核心代码包括：`ai_backend/services/decision_service.py`、`ai_backend/app.py`、`ai_backend/services/vision_service.py`、`ai_backend/services/speech_service.py`、`backend/api/services/AiUpdateService.php`、`frontend/assets/js/app.js`、`frontend/index.html`、`frontend/assets/css/style.css`。官方 PR 中还应包含 `README.md`、`requirements.txt`、`router.php`、`backend`、`frontend`、`ai_backend` 等运行所需文件。

十一、演示视频建议

视频必须控制在 5 分钟以内。建议结构：0:00-0:30 说明痛点和页面；0:30-1:30 展示 Camera 正常检测和低风险；1:30-2:20 展示 suspected fall 和风险上升；2:20-3:10 展示语音回答 我没事 后风险下降；3:10-4:00 展示 我需要帮助/Help me 后高风险和救援动作；4:00-4:50 展示 Video Upload 稳定检测；4:50-5:00 总结 Gemma 4 多模态决策和原生函数调用。

十二、结论

本项目以 Gemma 4 作为老人安全场景中的多模态决策层，将视觉检测、语音确认和无响应上下文转化为可执行的安全动作。系统不仅能展示跌倒检测，还能展示主动询问、风险解释和报警决策，符合 Gemma 4 大赛对核心代码、多模态处理、Native Function Calling 和真实痛点解决的要求。